

Objetivo:

Implementación de dos programas sencillos en C++, que utilizan un mismo TAD genérico agrupación, para gestionar una agenda de contactos y una lista de la compra de productos, respectivamente.

Descripción detallada:

- 1) Completa la implementación del TAD genérico agrupación definido a continuación, añadiendo al fichero *agrupacion.hpp* (facilitado como anexo) los comentarios necesarios, la implementación de las operaciones y las operaciones del iterador¹.

```
espec agrupaciones
usa booleanos
parámetro formal
  género elem
  operación
    iguales: elem e1, elem e2 -> booleano
    {Dados dos datos de tipo elem e1 y e2, devuelve verdad si y sólo e1 y e2 son
     considerados iguales, devuelve falso en caso contrario}

fpf
género agrupación {Los valores del TAD agrupación representan colecciones a las que
 se pueden añadir elementos de tipo elem, y de las que se pueden eliminar sus
 elementos de uno en uno, eliminándose siempre el último elemento añadido de todos
 los que contenga la agrupación}

operaciones
  iniciar: -> agrupación
  {Devuelve una agrupación vacía, sin elementos}
  añadir: agrupación a, elem e -> agrupación
  {Devuelve una agrupación igual a la resultante de añadir un elemento e en la
   agrupación a.}
  vacía: agrupación a -> booleano
  {Devuelve verdad si y sólo si la agrupación a está vacía}
  borrarUltimo: agrupación a -> agrupación
  {Si a no está vacía, devuelve una agrupación igual a la resultante de eliminar de a
   el último elemento añadido a ella. Si a está vacía, devuelve la agrupación vacía}
  está: agrupación a, elem e -> booleano
  {Dada una agrupación a y un elemento e, devuelve verdad si y sólo si en a hay algún
   elemento igual a e (en el sentido de la función iguales del parámetro formal elem),
   falso en caso contrario}

  {EN LA IMPLEMENTACIÓN, AÑADIR LAS OPERACIONES DE UN ITERADOR1}
```

fespec

- 2) Modifica el programa principal de la práctica 1 para utilizar el TAD genérico agrupación, particularizado para contactos, en lugar de utilizar la implementación que hiciste del TAD agenda en la práctica 1.
- 3) Implementa el TAD producto definido como:

```
espec productos
usa cadenas, enteros, booleanos
género producto {Los valores del TAD productos representan productos para los
 que se tiene información de su nombre y cantidad}

operaciones
  crear: cadena nom, entero c -> producto
  {Dada una cadena nom y un entero c, se obtiene un producto de nombre nom y con cantidad
   de producto c}
  nombre: producto p -> cadena
  {Dado un producto p, se obtiene la cadena correspondiente al nombre de ese producto}
  cantidad: producto p -> entero
  {Dado un producto p, se obtiene el entero que corresponde a la cantidad de p}
  iguales: producto p1, producto p2 -> booleano
  {Devuelve verdad si y sólo los productos p1 y p2 tienen el mismo nombre}
```

fespec

- 4) Modifica el programa principal de la tarea 2 añadiendo, además de lo que ya se hacía, lo que sea necesario para manejar una lista de la compra en la que:
 - a) se introduzcan varios productos en una agrupación de productos;
 - b) se muestren por pantalla los datos de todos los productos introducidos¹;
 - c) se borre el último producto, y se vuelvan a mostrar todos los datos¹;
 - d) dado el nombre de un producto, compruebe si en la agrupación hay algún producto con ese nombre;
 - e) ver qué ocurre si se compila código que intente introducir un contacto en la lista de la compra.

Instrucciones.

- El código fuente será compilado y probado en *lab000.unizar.es*, que es donde deberá funcionar correctamente.
- El código fuente deberá compilar correctamente con la opción `-std=c++11` activada.
 - Esto significa que, si se trabaja con la línea de comandos en *lab000*, deberá compilarse con (sin tener que indicar subdirectorios o rutas relativas para los ficheros):
`g++ -std=c++11 ficheros_compilar...`
 - Si se trabaja con algún entorno de programación (*Geany*, *Visual Studio Code*, *Code::Blocks*, *CodeLite*, etc) y no se utiliza la línea de comandos para compilar, el proyecto de la práctica deberá estar configurado para compilar con la opción `-std=c++11`.
- Todos los ficheros con código fuente deberán estar **correctamente documentados**.
- En el comentario inicial de **cada fichero** de código fuente se añadirán los **nombres completos y NIP de los autores**.
- Los TADs deberán implementarse siguiendo las instrucciones dadas en las clases y prácticas de la asignatura, y no se permite utilizar Programación Orientada a Objetos.
- No se permite usar las clases o componentes de la *Standard Template Library (STL)*, ni similares.

¹ El texto subrayado (implementación de un iterador y muestra en pantalla de todos los datos) lo podréis realizar después de que veamos en clase el concepto de iterador.